Assignment 2 – Summary

GitHub username	ishan-monie
Repository name	advanced-ml-hw2
Link to the code	https://github.com/ishan-monie/advanced-ml-hw2

All the code is in this repository (Python 3.12). One Deep Q-Network, in `src/dqn.py`, is used for both games.

How to run the code

First install the packages with `pip install -r requirements.txt`, then run the commands below from the repository folder. The checks in `tests/test_dqn.py` print PASS or FAIL for each check, with the numbers behind it.

What	Command
Train CartPole (about 3 minutes on a CPU)	`python src/dqn.py --game cartpole --seed 0 --out runs/cartpole_seed0 --snapshots 10,30,60,100,200,400`
Train MsPacman (about 3 hours on one GPU)	`python src/dqn.py --game pacman --seed 0 --device cuda --out runs/pacman_seed0`
Play the 500 test games	`python src/evaluate_models.py --game cartpole --model runs/cartpole_seed0/model_best.pt --out results/cartpole_eval.npz` `python src/evaluate_models.py --game pacman --model runs/pacman_seed0/model_best.pt --out results/pacman_eval.npz --device cuda`
Make all the graphs (figures/)	`python src/make_plots.py`
Run the checks (about 5 minutes)	`python tests/test_dqn.py`
Extra CartPole experiments	`python experiments/sweep_cartpole.py`, `python experiments/diagnose_cartpole.py`, `python experiments/experiment_discount.py`

The finished runs, networks and test scores are saved in `runs/` and `results/`, so the graphs and checks work without training again. Seeds are fixed. CartPole runs use 1 CPU thread, because a different thread count can change a run.

What is in the repository

File or folder	What it holds
`src/dqn.py`	The Deep Q-Network: replay buffer, networks, target network, training loop and test games.
`src/make_plots.py`, `src/evaluate_models.py`	All graphs, including the two non-standard ones (`figures/`).
`tests/`	The edge-case and behaviour checks.
`experiments/`	CartPole tuning, why some networks failed, and the discount experiment (results in the `.csv` files).
`runs/`, `results/`	Training logs, saved networks and test scores.

Deep Q-Network

(a) CartPole-v1

Assumptions

- Network: the 4 state numbers go into two hidden layers of 128 units, and the output is 2 Q values (push left, push right). Discount 0.95, Adam with learning rate 5e-4, Huber loss and Double DQN.
- Replay buffer: the last 50,000 steps. Learning starts after 1,000 steps, then there is one update per step on a random batch of 64. The target network is a copy of the network that is refreshed every 250 steps.
- Epsilon schedule: the chance of a random push goes from 1 down to 0.02 over the first 10,000 steps. Training lasts 800 games.
- Choosing the network: the score jumps around a lot from game to game, so every 10 games the network plays 30 test games from fixed starting states, and the best one is kept. These are not the final 500 test games, which use a different random seed and no random pushes.
- A game stopped by the 500-step limit is not counted as game over, because the pole would have kept scoring. The main model is seed 0, picked in advance. Seeds 1 to 4 show how much luck matters.

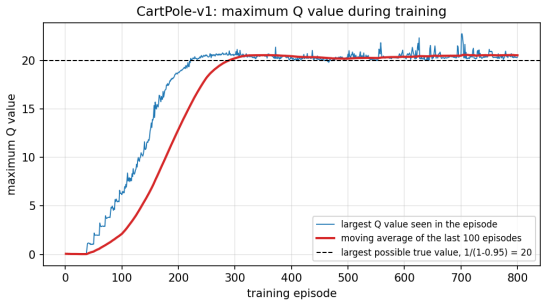


Figure 1. Largest Q value in each training game (blue) and its 100-game moving average (red). The true value of a balanced pole is at most 20.

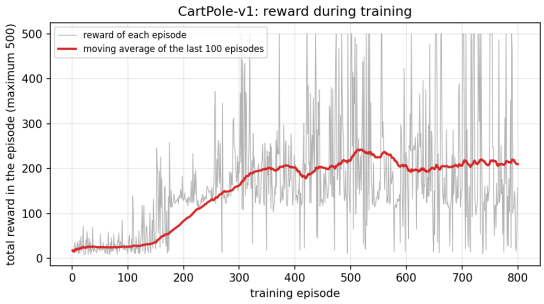


Figure 2. Reward of each training game and the 100-game moving average.

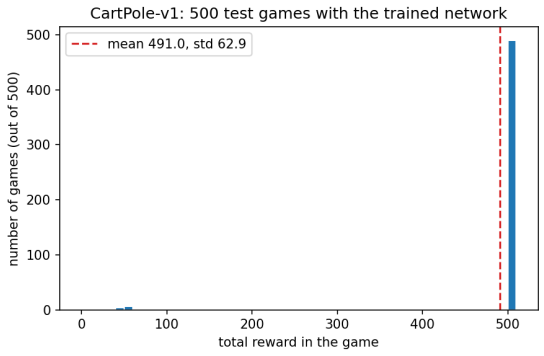


Figure 3. 500 test games with the chosen network (seed 0): mean 491.0, standard deviation 62.9.

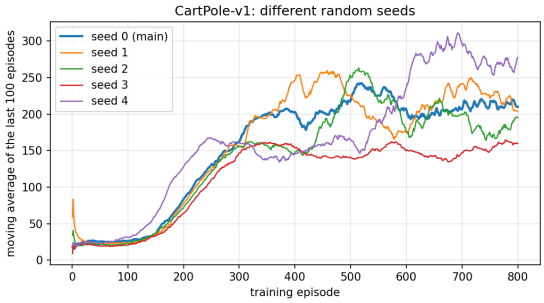


Figure 4. The five training seeds (100-game moving average of the training reward).

The network solves the game. On the 500 test games the mean was 491.0 (standard deviation 62.9) and 98% of the games lasted the full 500 steps. The other seeds scored 498, 498, 378, 500. The largest Q value climbs to about 20. This is the most a value can be, because every step gives 1 and 1/(1-0.95) = 20. It goes a little above 20 during training, which is the usual over-estimation in Q learning. The training curve (Figure 2) only averages about 210, because training games include random pushes and the network keeps changing. The network we keep plays much better than that, and the network at the very end of training plays much worse (see the tests). That is why the network is chosen by test play.

Non-standard plot: CartPole policy map

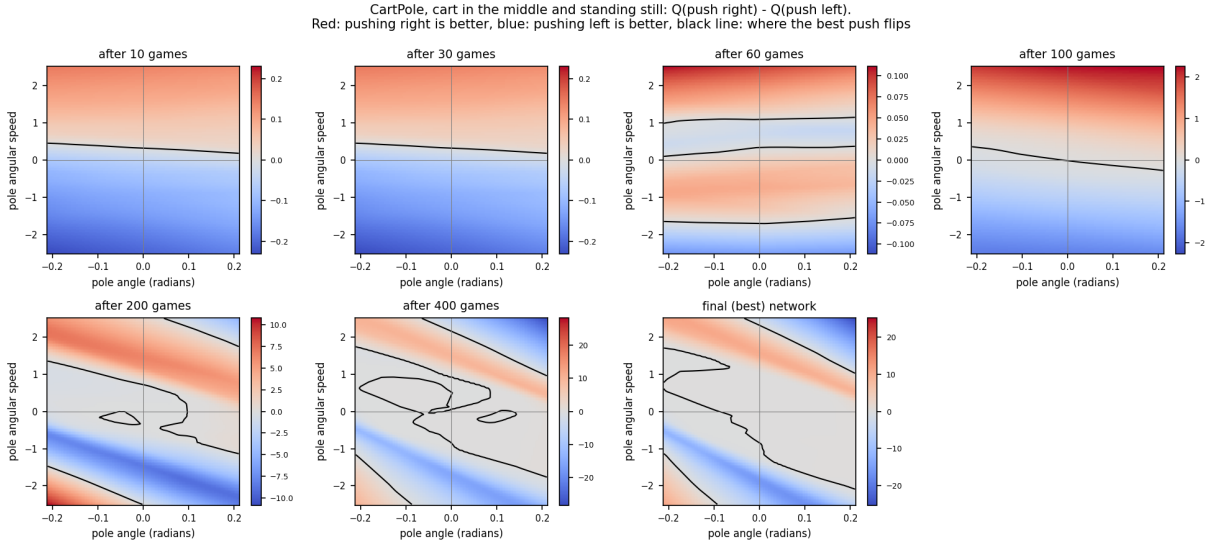


Figure 5. For each pole angle and pole speed (cart in the middle and not moving), the Q value of pushing right minus pushing left, after 10, 30, 60, 100, 200 and 400 games and for the chosen network. After 10 and 30 games the network only reacts to the pole speed. By 200 to 400 games it has learned that a fast-falling pole needs a push the other way.

Tuning

Our first CartPole runs only scored 100 to 250. We tried 11 settings with 3 seeds each (learning rate, target update, buffer size, extra game-over steps in every batch, a falling learning rate, a bigger network). Then we looked at how the games were lost. In 2,013 of 2,262 lost games (from 25 networks taken at the end of training) the cart left the track and the pole had not fallen. A discount of 0.95 only looks about 20 steps ahead, so a slow drift toward the edge is hard to notice. What helped was choosing the network by test play instead of by the training average. The table shows the average score on 100 test games.

Setting	Network at the end, round 1 (3 seeds)	Chosen by training average, round 1	Network at the end, round 2 (5 seeds)	Chosen by test play, round 2
hard target update every 500 steps	248	256	193	439
soft target update (the first try)	181	248	120	485
hard target update every 250 steps (used)	not run	not run	145	486
plus 25% game-over steps in every batch	294	184	201	429

Setting	Network at the end, round 1 (3 seeds)	Chosen by training average, round 1	Network at the end, round 2 (5 seeds)	Chosen by test play, round 2
plus a falling learning rate (1e-3 to 1e-5)	256	226	249	468

Once the network is chosen by test play, the settings are about equally good (the same setting scores anywhere from 261 to 500 depending on the seed). The one with the highest average, a hard target update every 250 steps, was used for the final runs on 5 other seeds.

Edge cases

Edge case	How it was handled	How it was checked
Time limit versus a real failure: after 500 steps the pole would have kept scoring.	Only a real failure is stored as game over. A time-limit stop still uses the value of the next state.	With a time limit of 30 steps, games stopped by the limit were stored as not over (8 games) and games where the pole fell as over (52 games).
The network keeps changing: the network at the end of training is much worse than earlier ones.	The network is chosen by 30 test games every 10 games, from fixed starting states that are not the final test games.	Seed 0: the chosen network scored 500 on 100 games, the last network 86.
Over-estimation: Q learning tends to think states are worth more than they are.	Double DQN, Huber loss, and a target network that only changes every 250 steps.	The largest Q value settles at 20.5 against a true maximum of 20 (highest 22.7). On the test games the average prediction was 19.95 against 19.99 actually earned.
The target network: learning breaks if it moves with the network or never moves.	It is a frozen copy refreshed every 250 steps and never trained directly.	It stayed the same during 25 learning steps and matched the network after a refresh. A version that never refreshed was caught on a tiny problem (error 1.05).
A discount of 0.95 only sees about 20 steps ahead, so drifting off the track is hard to learn.	We tried a falling learning rate, extra game-over steps in every batch, and a longer view (discount 0.99).	The cart left the track in 2,013 of 2,262 lost games (25 networks from the end of training). With discount 0.99 this fell from 473 to 279 games, and 102 games reached 500 (0 with 0.95).
Results that cannot be repeated.	Seeds are fixed, runs use 1 thread, and the test games have their own seed.	The same seed gave an identical run and a different seed a different one. The saved network replays the saved test scores exactly.

Tests

Test	What was expected	What happened
The number each Q value is pulled toward (reward + discount x value of the next state), worked by hand for 3 steps, the last one a game over, with and without Double DQN.	Code and hand calculation agree.	They matched (0.892, -0.259, and exactly the reward, 5, for the game-over step).
The first 10 steps of a real game, looked at one by one.	Start state near 0, reward 1 every step, each next state equal to the following state, only pushes 0 or 1.	All true (first state 0.014, -0.023, -0.046, -0.048).
A tiny problem with a known answer: a chain of 5 squares solved exactly, then learned by the same code.	The learned Q values match the exact ones.	Largest difference 0.000.
Can that test fail? A target network that is never updated, and a discount of 1 instead of 0.9.	Both should be caught.	Errors of 1.05 and 0.35 against 0.000.
Change the problem: discount 0.	Every CartPole state is worth just the next reward, 1.	All 4,000 Q values were between 0.989 and 1.033.
Replay buffer: when full, the newest steps stay, and batches are drawn evenly from the whole buffer.	Only the newest 10 of 25 steps remain, and the age of drawn steps is spread evenly.	Newest 10 kept. Average age 2,503 of 5,000 (even spread: 2,500), KS test p = 0.91.
Trained network against random play (500 and 100 games).	Much better (statistical test, t above 10).	491.0 against 23.9, t = 151.
Predicted against actual value in the test games.	Within 5% on average.	19.95 predicted, 19.99 actual.
Luck: 5 seeds.	All well above random, most near 500.	Test means 491, 498, 498, 378, 500.
Change the problem: discount 0.99 instead of 0.95 (5 seeds, network at the end of training).	We predicted the average score would be 1.5 times higher. That was wrong (the network still keeps changing), but fewer games were lost by leaving the track.	Average 166 against 145. Cart left the track 279 times against 473. Games reaching 500: 102 against 0.

All 25 checks for this game (16 general ones and 9 on the saved CartPole runs) are in tests/test_dqn.py and tests/test_results.py.

With 3 more months: CartPole

The score depends on which network is kept, so we would first find out why the network keeps changing (learning rate, target update interval, batch size) and try more stable methods such as prioritised replay, n-step returns or a dueling network. We would also use 20 or more seeds with confidence ranges, test the trained network with a heavier or longer pole, measure over-estimation against exact values on small versions of the problem, and see how much delay or noise in the state the network can handle.

(b) MsPacman-v5

Assumptions

- Input: the given `preprocess_observation` turns each screen into an 88 x 80 picture. The network sees the last 4 pictures stacked together, so it can tell which way things are moving. At the start of a game the first picture is repeated. The game (v5) repeats our previous action 25% of the time, skips 4 frames per step, has 9 actions, and ends when the last life is lost. Losing one life does not end the game for the network.
- Network: three convolution layers (32 filters of 8 x 8 stepping 4, 64 of 4 x 4 stepping 2, 64 of 3 x 3) and a hidden layer of 512 units, with 9 outputs, one Q value per action (about 1.5 million weights). Discount 0.99, Adam with learning rate 1e-4, Huber loss, Double DQN, and gradient size limited to 10.
- Replay buffer: the last 100,000 steps (stored as small whole numbers, about 5.6 GB). Learning starts after 10,000 steps, then one update on a random batch of 32 every 4 steps. The target network is a copy refreshed every 2,000 steps.
- Epsilon schedule: the chance of a random action goes from 1 down to 0.05 over the first 400,000 steps (about 800 games) and then stays at 0.05.
- Rewards: a pellet is worth 10 points and a ghost 200 or more, so for learning every reward is clipped to between -1 and +1. The scores in the graphs are the real game scores. The Q values in Figure 6 are in clipped units, where the highest possible value is 1/(1-0.99) = 100.
- Training: 6,000 games (about 4.6 million steps, 3.9 hours on one GPU of the course server). The network that is kept is the one with the best 100-game moving average of the training reward. The test games play no part in that choice. The 500 test games use their own random seed and a 5% chance of a random action (a network that is always greedy can get stuck repeating moves), and were played on the CPU. The main model is seed 0, picked in advance. Seed 1 shows how much luck matters.

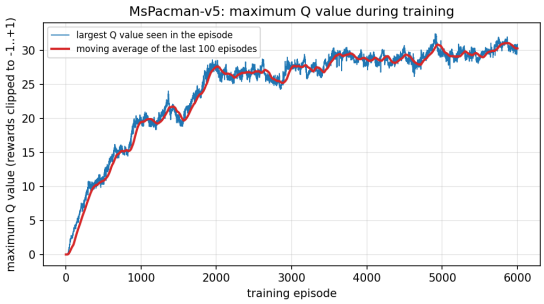


Figure 6. Largest Q value in each training game (blue) and its 100-game moving average (red), in clipped-reward units.

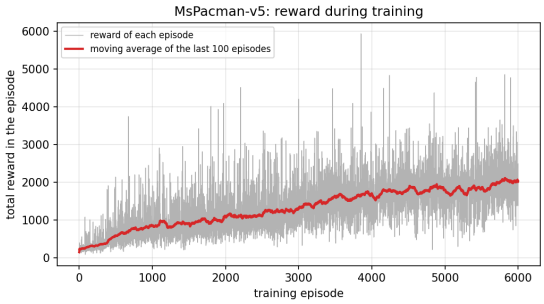


Figure 7. Score of each training game and the 100-game moving average.

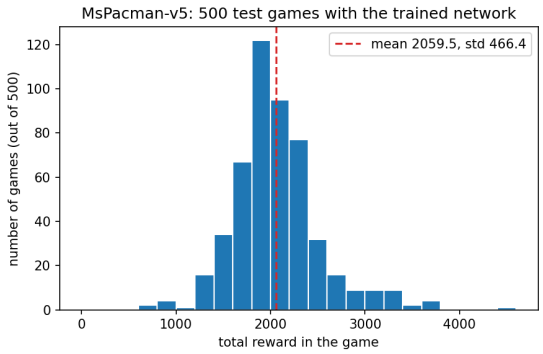


Figure 8. 500 test games with the chosen network (seed 0): mean 2059, standard deviation 466.

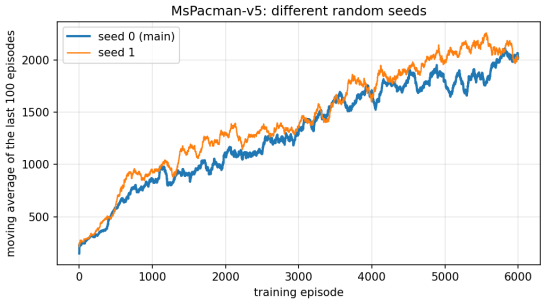


Figure 9. The two training seeds (100-game moving average of the training score).

The network improves steadily during training. The 100-game average goes from 274 in the first 100 games to 857 after 1,000, 1294 after 3,000 and 2024 at the end (seed 1: 286 to 2013). It is still rising slowly at 6,000 games, so more training would help. On the 500 test games the chosen network scored a mean of 2059 (standard deviation 466, lowest 680, highest 4530). The second seed scored a mean of 2088 (568). The largest Q value rises from about 0 to about 31, far below the maximum of 100. That makes sense, because a pellet only gives 1 clipped point.

Non-standard plot: predicted against actual value

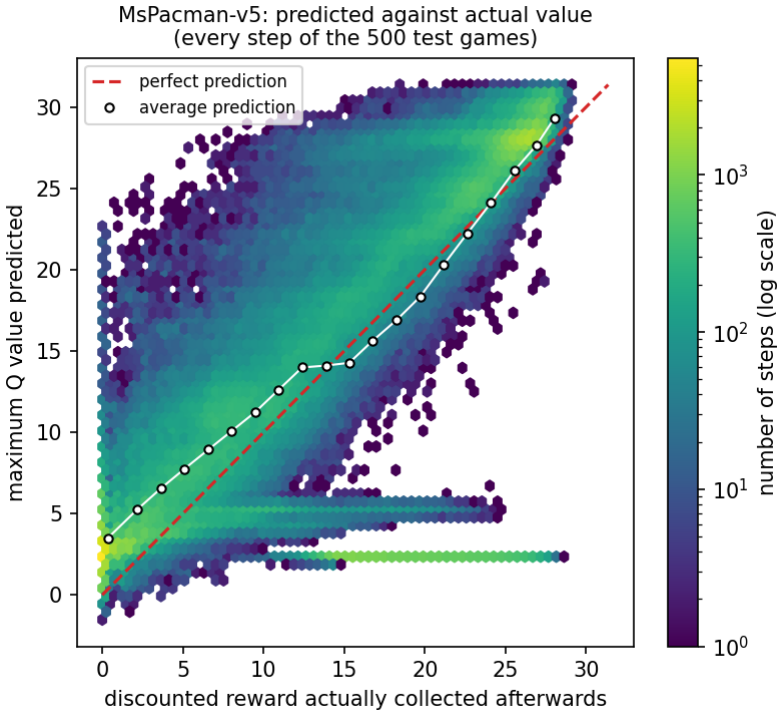


Figure 10. For every step of the 500 test games: the largest Q value the network predicted (vertical) against the discounted reward actually collected afterwards (horizontal), in clipped-reward units. On the dashed line the prediction would be exactly right. White dots are averages. The network over-estimates when little reward follows and under-estimates when a lot follows. This is normal, because it cannot know in advance which games will go well.

Edge cases

Edge case	How it was handled	How it was checked
Rewards of very different sizes (10 points for a pellet, 200 or more for a ghost).	Rewards are clipped to between -1 and +1 for learning. Reported scores are the real ones.	Largest stored reward 1.0, and real game scores are multiples of 10 (checked on 2 games).
One picture cannot show which way things move, and pictures from two games must not mix.	Four pictures are stacked. A new game starts with the first picture repeated 4 times.	A new game started with 4 identical pictures, each step moved the stack by exactly one picture, and the game's own screen was left unchanged.
Randomness in the game (it repeats the previous action 25% of the time), and a greedy network getting stuck.	Test games use a 5% chance of a random action and their own seed. They were played on the CPU so they can be replayed exactly.	The first 3 saved test scores were reproduced exactly from the saved network.
Over-estimation and Q values growing without limit.	Double DQN, Huber loss, a target network refreshed every 2,000 steps, and clipped rewards.	The largest Q value stayed near 31 (highest ever 32), far below 100.
Exploration: too little early on, or too much late.	The chance of a random action falls in a straight line from 1 to 0.05 over 400,000 steps and then stays at 0.05.	First game 0.999. The minimum was reached at step 400,000 and kept to the last game.
A very large replay buffer and a very long run.	Pictures are stored as small whole numbers. The log and the network are saved every 25 games, and a time limit stops a run in time.	The buffer holds 100,000 steps (5.6 GB). Both runs finished 6,000 games in 3.9 and 3.9 hours.

Tests

Test	What was expected	What happened
Pictures: the cleaned frame is 88 x 80 with values from -128 to 127, a new game starts with the first frame repeated 4 times, each step slides the stack by one frame, and there are 9 actions.	All true.	All true.
Rewards: stored values between -1 and +1, reported scores real.	Largest stored reward 1, scores multiples of 10.	Largest stored reward 1, scores 150 and 160.
Trained network against random play (500 and 60 games).	Much better (statistical test, t above 10).	2059 against 258, t = 69
Can the results fail? A network with random weights.	It should score like random play, far below the trained network.	untrained network 384 (20 games), trained network 2059
Learning: the last 100 training games against the first 100.	At least 3 times higher, and not by luck.	first 100 games 274, last 100 games 2024, p = 6.0e-61
Luck: the second training seed.	It also learns (at least 3 times higher).	seed 0: 274 -> 2024; seed 1: 286 -> 2013
Epsilon schedule.	Starts near 1, reaches 0.05 only after 400,000 steps and stays there.	As expected.
Predicted against actual value in the test games.	The predictions follow the actual returns (correlation above 0.5).	correlation 0.79; average prediction 14.4, average actual 13.3; highest prediction 31.4
The saved network replays the saved test scores exactly (first 3 games).	Identical.	replayed [1620.0, 2520.0, 1700.0], saved [1620.0, 2520.0, 1700.0]

The 9 checks on the saved MsPacman runs (and the 16 general ones shared with CartPole) are in tests/test_dqn.py and tests/test_results.py. All 34 checks in the file pass.

With 3 more months: MsPacman

The score was still rising when training stopped, so the first step would be much longer training with several seeds and confidence ranges. We would add the usual Atari improvements (prioritised replay, n-step returns, a dueling network) one at a time to see what each is worth, count losing a life as game over, and compare with the scores of random play and of a human player. We would also check how much the 4-picture stack and the reward clipping matter, test the network on different random starts and with the 25% repeated actions switched off, and look at what the network pays attention to in a picture.